Фокус на трех вещах.

Первое - понимание логики работы системы: методы, данные, правила обработки, интеграции. Чтобы мы могли уверенно ответить на вопрос «где и почему это сломалось».
Второе - peer review как инструмент роста, а не контроля. Мы будем усиливать друг друга и вместе делать более качественные решения.
Третье - работа с инцидентами на уровне причин и архитектурных паттернов, а не только фиксов.

Слой	Скилл
Логика	методы, обработка, ошибки
Данные	таблицы, связи, статусы
Интеграции	API, контракты
Middleware	прокси, шлюзы, деплой
Инциденты	RCA, устойчивость
Архитектура	системное мышление

ЛОГИКА ОБРАБОТКИ (design-level)

То есть:

- какие шаги обработки существуют
- в каком порядке
- какие условия переходов
- какие статусы возможны
- какие ошибки допустимы
- что считается успешным результатом
- что делать при частичном успехе

Где разбираемся

- методы и обработка запросов;
- бизнес-правила и ФЛК;
- таблицы, статусы, жизненный цикл сущностей;
- где именно принимаются решения в системе.

В течение года у нас появятся общие задачи развития системного анализа, под которые каждый аналитик сможет выделять до 20% рабочего времени. Это часть нашей профессии и нашего роста, а не дополнительная нагрузка.

- понимать, **в каком методе ломается логика**
- описывать правила обработки
- читать код и говорить с backend на одном языке

Основы разработки

- функции
- аргументы
- возврат значений
- ветвления (if / else)
- логические выражения

Углублённый

- области видимости
- исключения
- обработка ошибок
- декораторы (очень важно для понимания middleware)

ООП

- классы
- методы
- инкапсуляция
- наследование

Отладка программ

- дебаггинг
- ошибки и баги
- stack trace

Это слой:

«Почему метод вернул ошибку»
«Где сломалась бизнес-логика»
«Почему данные не дошли дальше».

① Общий инженерный язык

Если аналитики говорят:

- таблицы
- ключи
- транзакции
- idempotency
- race condition
- consistency
- retries
- timeout
- saga
- compensation
- eventual consistency

Это первое, что создаёт уважение внутри команды.

② Общая инженерная боль

Людей объединяют **не задачи**, а **одинаковые проблемы**:

- “почему это упало?”
- “почему снова тот же инцидент?”
- “почему API ведёт себя по-разному?”
- “почему данные разъехались?”
- “почему бизнес думает одно, а система делает другое?”

Когда аналитик участвует в разборе таких вещей —
он чувствует **реальное влияние**.

③ Общий рост

Если аналитик понимает: “Здесь я стану сильнее как инженер мышления” он остаётся даже без денег. Если не понимает — уходит даже с деньгами.

Какие темы реально прокачивают CA

Ты правильно чувствуешь:
не весь бэкенд нужен аналитикам.

Что НЕ нужно СА

Не надо:

- писать Django
- писать Flask
- писать CRUD
- учить HTML/CSS
- писать тесты руками
- деплоить Docker

Это создаёт иллюзию развития, но не влияет на мышление.

Что РЕАЛЬНО прокачивает системного аналитика

Я выделю 6 инженерных слоёв.

Уровень 1. Data thinking (база)

То, что напрямую связано с их работой:

- как данные рождаются
- где они хранятся
- как трансформируются
- кто владелец
- кто меняет
- кто читает

Темы:

- таблица ≠ сущность
- справочник ≠ реестр
- master-data vs operational-data
- источник истины
- жизненный цикл записи
- статус как атрибут системы

Уровень 2. Алгоритмическое мышление

- сложность алгоритмов (O(n))
- почему «плохо написанный процесс» = дорогая система
- очереди
- стеки
- хеш-таблицы

- поиск
- сортировки (на уровне идеи)

❗ НЕ для кода а для понимания: “почему система тормозит, ломается и масштабируется плохо”. Это **уровень архитектурного мышления**, не программирования.

Уровень 3. API-мышление

Аналитик обязан понимать:

- API-first
- контракт важнее реализации
- backward compatibility
- versioning
- breaking change
- idempotency
- retries
- timeout
- error handling

Это напрямую ложится на твою инициативу:

- автогенерация API
- качество интеграций
- меньше инцидентов

Уровень 4. Системные сбои и инциденты

Темы:

- почему инцидент — это не баг
- типовые причины падений
- race condition
- duplicate request
- lost message
- partial failure
- distributed systems failures

Это превращает CA в **архитектурного партнёра**, а не документатора.

Уровень 5. Архитектурные паттерны

Без кода. Только мышление.

- sync vs async
- event-driven
- saga
- compensation
- retry pattern
- circuit breaker
- cache aside
- outbox

Именно здесь аналитики начинают **восхищаться лидером**, который это объясняет.

Уровень 6. Системное мышление

- система ≠ набор задач
- система = связи
- локальная оптимизация ломает целое
- метрика искажается
- человек влияет на систему
- процесс рождает поведение

Chief Systems Thinking Engineer

Человеком, который:

- видит систему целиком
- понимает логику кода
- понимает логику данных
- понимает логику бизнеса
- соединяет всё в одно мышление

> В этом году я хочу, чтобы мы перестали быть разрозненными аналитиками в разных продуктах и начали формировать инженерное сообщество системного анализа.
>
> Мы не будем учиться писать код.
> Мы будем учиться понимать, как работают системы.
>
> Наша цель — чтобы системный аналитик в банке:
>
> - понимал данные

- понимал интеграции
- понимал причины инцидентов
- понимал архитектурные паттерны

Я, как Хэд системного анализа, беру на себя задачу выстроить для команды программу инженерного мышления — через воркшопы, реальные кейсы банка и совместные разборы.

Это развитие не ради галочек, а ради того, чтобы мы стали сильнее как команда и как инженеры мышления.

6 инженерных слоев системного анализа

1️⃣ Слой логики обработки (методы, правила, ФЛК)

👉 **Где система принимает решения**

Что должен уметь СА:

- понимать, **в каком методе ломается логика**
- описывать правила обработки
- читать код и говорить с backend на одном языке

Темы курса:

📘 **Основы Python**

- функции
- аргументы
- возврат значений
- ветвления (if / else)
- логические выражения

📘 **Углублённый Python**

- области видимости
- исключения

- обработка ошибок
- декораторы (очень важно для понимания middleware)

📘 **ООП**

- классы
- методы
- инкапсуляция
- наследование

📘 **Отладка программ**

- дебаггинг
- ошибки и баги
- stack trace

💡 **Это слой:**

> «Почему метод вернул ошибку»,
> «Где сломалась бизнес-логика»,
> «Почему данные не дошли дальше».

2️⃣ Слой данных (таблицы, состояния, целостность)

👉 **Где живёт правда системы**

Что должен уметь СА:

- понимать структуру таблиц
- видеть причину ошибок на уровне БД
- анализировать неконсистентность данных

Темы курса:

📘 **Базы данных**

- SQL
- SELECT / JOIN
- агрегаты
- группировки

📘 **Связи таблиц**

- one-to-one

- one-to-many
- many-to-many
- ссылочная целостность

📘 **ORM (Django ORM)**

- CRUD
- фильтрация
- связанные модели

📘 **Оптимизация запросов**

- N+1 проблема
- производительность

💡 **Это слой:**

> «Почему данные есть в одной таблице, но нет в другой»,
> «Почему статус не обновился»,
> «Почему отчёт показывает ерунду».

3️⃣ Интеграционный слой (API, контракты, обмен)

👉 **Где системы разговаривают друг с другом**

Это твой ключевой слой.

Темы курса:

📘 **Что такое API**

- HTTP
- запрос / ответ
- форматы JSON
- REST

📘 **REST API**

- ресурсы
- методы (GET / POST / PUT / DELETE)
- эндпоинты

📘 **Django Rest Framework**

- сериализаторы

- вьюхи
- вьюсеты
- маршрутизация

📘 **JWT**

- токены
- авторизация
- срок жизни токена

📘 **Документация API**

- структура API
- описание контрактов

💡 **Это слой:**

> «Почему сервис А не получил ответ от сервиса Б»,
> «Почему упал 401 / 403»,
> «Почему фронт получает не тот JSON».

4️⃣ Middleware слой (шлюзы, прокси, инфраструктура)

👉 **Между сервисами**

Это то, чего почти никогда не хватает системным аналитикам.

Темы курса:

📘 **Nginx**

- reverse proxy
- маршрутизация
- HTTPS

📘 **Docker**

- контейнеры
- сети
- взаимодействие сервисов

📘 **Docker Compose**

- несколько сервисов
- связи между ними

📘 **CI/CD**

- пайплайны
- деплой
- сборка

📘 **CORS**

- политики доступа
- ошибки браузера

💡 **Это слой:**

> «Почему запрос не дошёл до сервиса»,
> «Почему работает локально, но не в тесте»,
> «Почему API открыто, но не вызывается».

5️⃣ Слой надёжности и инцидентов (RCA)

👉 **Почему система упала**

Темы курса:

📘 **Логирование**

- журнал ошибок
- исключения

📘 **Тестирование**

- unit-тесты
- интеграционные тесты

📘 **Мониторинг**

- ошибки
- доступность

📘 **Алгоритмы и структуры данных**

- производительность
- сложность
- узкие места

📘 **Асинхронность**

- блокировки
- ожидания
- параллельность

💡 **Это слой:**

> «Почему сервис лёг под нагрузкой»,
> «Почему ошибка повторяется»,
> «Почему RCA без корня причины».

6️⃣ Архитектурный слой (мышление системы)

👉 **Почему система вообще такая**

Темы курса:

📘 **Архитектура REST**

- API-first

📘 **DDD (частично)**

- домены
- границы

📘 **Асинхронность**

- события
- очереди (логика)

📘 **Алгоритмы**

- эффективность
- масштабируемость

📘 **Проектирование архитектуры (AI-модуль)**

- декомпозиция
- техдолг

💡 **Это слой:**

> «Почему мы так построили систему»,
> «Где граница сервиса»,
> «Почему нельзя просто добавить ещё один метод».

Итоговая карта

Инженерный слой	Что прокачивает
Логика	методы, обработка, ошибки
Данные	таблицы, связи, статусы
Интеграции	API, контракты
Middleware	прокси, шлюзы, деплой
Инциденты	RCA, устойчивость
Архитектура	системное мышление

🧭 Roadmap развития команды системной аналитики (6 месяцев)

Цель программы

Сформировать команду системных аналитиков, которая:

- понимает **как реально работает система**, а не только требования;
- умеет участвовать в RCA и архитектурных разборах;
- снижает количество инцидентов и переделок;

- усиливает продуктовые команды инженерной экспертизой.

🔹 МЕСЯЦ 1 — БАЗА: как работает система

Фокус

Логика обработки и данные

Что прокачиваем

- методы и обработка запросов;
- бизнес-правила и ФЛК;
- таблицы, статусы, жизненный цикл сущностей;
- где именно принимаются решения в системе.

Форматы

- 2 коротких воркшопа (30–40 мин)
- 1 разбор реального кейса банка
- простые схемы “запрос → обработка → данные → ответ”

Результат

СА начинают понимать:

> где реально «живёт логика», а не только требования.

🔹 МЕСЯЦ 2 — ДАННЫЕ И СОСТОЯНИЯ

Фокус

Истина системы = данные

Что прокачиваем

- master data vs operational data;
- статусы и переходы;
- источники истины;
- почему данные «разъезжаются».

Форматы

- схемы таблиц;
- трассировка данных;
- кейсы «почему статус не обновился».

Результат

СА перестают мыслить экранами и начинают мыслить данными.

🔹 МЕСЯЦ 3 — ИНТЕГРАЦИИ И API

Фокус

Как системы общаются друг с другом

Что прокачиваем

- REST / HTTP / JSON;
- контракт важнее реализации;
- ошибки 4xx / 5xx;
- идемпотентность;
- версии API.

Форматы

- разбор наших API;
- чтение OpenAPI;
- peer review API-контрактов.

Результат

СА уверенно участвуют в проектировании интеграций.

🔹 МЕСЯЦ 4 — MIDDLEWARE И ИНФРАСТРУКТУРА

Фокус

Что находится между сервисами

Что прокачиваем

- API Gateway;
- прокси;
- маршрутизация;
- окружения (dev / test / prod);
- почему запрос не дошёл.

Форматы

- схемы потоков запросов;
- реальные кейсы “работает локально — не работает в тесте”.

Результат

CA начинают понимать инфраструктурные причины ошибок.

🔹 МЕСЯЦ 5 — ИНЦИДЕНТЫ И RCA

Фокус

Почему система падает

Что прокачиваем

- типовые причины инцидентов;
- частичные сбои;
- повторяемость ошибок;
- архитектурные анти-паттерны.

Форматы

- участие CA в RCA;

- совместные разборы;
- формирование гипотез.

Результат

СА перестают быть наблюдателями инцидентов.

🔷 МЕСЯЦ 6 — АРХИТЕКТУРНОЕ МЫШЛЕНИЕ

Фокус

Система целиком

Что прокачиваем

- границы сервисов;
- ответственность компонентов;
- компромиссы решений;
- последствия архитектурных выборов.

Форматы

- архитектурные разборы;
- ADR;
- обсуждение альтернатив решений.

Результат

СА начинают мыслить последствиями, а не задачами.

🔸 Кейсы банка

- без абстракций;
- только реальная практика.

🎯 Что будет через 6 месяцев

- аналитики понимают систему глубже;
- сильнее диалог с backend и архитекторами;
- меньше повторяющихся ошибок;
- выше качество требований;
- появляется единая команда CA, а не разрозненные специалисты.